第16天 · 失败以后，怎样重试才合理

区分暂时故障、参数错误与时间预算，避免无限重试。

标准路线：60分钟；忙碌日：20分钟；扩展练习另计。

今天的目标

- 解释总尝试次数与成功前失败次数的关系。

- 运行一个会成功和一个会耗尽预算的重试案例。

- 区分超时返回、取消工作与重复副作用。

开始之前

- 已经完成第 15 课，或能解释其产物。

- 打开实践包 practice 目录；后文 python 可替换为虚拟环境内解释器的完整路径。

一小时安排

- 15分钟 · 理解概念：阅读本课四节讲解，弄清一个关键机制与适用边界。

- 30分钟 · 亲手操作：先跑基线，再做一次单变量实验，保留前后结果。

- 10分钟 · 检查结果：逐项验收，完成两道自测并阅读解析。

- 5分钟 · 留下记录：写出本课发现、未解决问题和下次第一步。

01 · 理解概念

先判断失败属于哪一类

读取服务偶尔超时，与把字符串传给整数参数不是同一种错误。暂时故障可能在下一次调用时消失；参数错误通常会原样重复。把所有异常都放进无限重试循环，会消耗时间，也会掩盖真正的错误。

本课用可控制的失败次数制造暂时故障。failures_before_success 表示成功前会失败几次，max_attempts 是总尝试次数，包含第一次。先比较这两个数，再预测程序应该成功还是耗尽预算。

重试次数必须有明确含义

当成功前失败2次，总尝试上限为3时，第三次才有机会成功。若把总尝试数改成2，程序应报告失败而停止。这里训练的是边界意识，不是猜一个“足够大”的数字。

生产服务还要考虑退避、抖动、服务端提示和哪些错误可重试。本课使用小规模受控延迟便于观察，不把这个教学策略包装成适用于所有网络服务的通用配置。

时间预算与尝试次数分开控制

尝试次数限制防止不停调用；截止时间限制一次工作等待多久。work_delay_ms 与 deadline_ms 用毫秒表示，所以改变时间参数前先核对单位。操作没有报错，也可能因超时而无法交付可用结果。

某些线程超时只意味着调用方停止等待，并不自动杀死底层工作。阅读程序实现时要区分“返回超时状态”和“工作已被彻底取消”；真实进程或远端请求需要相应取消机制。

重试可能重复产生副作用

检索同一份材料通常可重复执行；发送邮件、写入报告或提交任务若被重复执行，可能产生两份结果。不要因为第一次请求没有返回，就假定第一次一定没有发生。

下一课会给写操作加入内容摘要、人工批准与幂等检查。本课先在笔记标出自己的流程中哪些动作只读、哪些会改变状态，并说明哪些允许直接重试。

02 · 一步一步做

命令在解压后的 practice 文件夹执行。Windows可使用 .\.venv\Scripts\python.exe；macOS/Linux可使用 .venv/bin/python 替代 python。

1. 找到本课的输入与配置

为什么做：先知道程序读什么，后面才能解释输出为何改变。

在 VS Code 中打开 experiments.json，定位 day16。用下面的命令确认当前目录正确，并把各参数的作用写在笔记中。

```
python -c "import json; print(json.load(open('experiments.json', encoding='utf-8'))['day16'])"
```

预期结果：终端显示本课配置；键名与文件中的内容一致。

2. 运行本课基线

为什么做：保持默认设置的一次运行，是后续比较的参照。

执行命令，打开新生成的 outputs/day16.json。先检查 day、mode 和 result，不急着修改代码。

```
python run.py day 16
```

预期结果：结果写入 outputs/day16.json；主线 mode 明确为 offline，代表规则教学流程。

3. 沿输出检查机制

为什么做：不能只看程序有没有报错，要检查它实际做了什么。

格式化阅读输出，重点检查retry.attempts、retry.status 和 deadline.status。把一个输入参数和一个输出字段用箭头对应起来。

```
python -m json.tool outputs/day16.json
```

预期结果：能够定位retry.attempts、retry.status 和 deadline.status，并用自己的话解释该结果。

4. 改变一个条件并比较

为什么做：主动制造可解释的变化，才能验证自己理解的机制。

完成下面“改一改”实验。对比基线与新结果，记录哪些字段改变、哪些保持稳定；不要同时改模型、材料和参数。

预期结果：留下一组前后结果和一句解释；失败状态也可以是正确的实验产物。

改一改 · 让成功发生之前，重试预算先耗尽

- 先执行 python run.py day 16，将 outputs/day16.json 复制为自己的基线备份。

- 在 VS Code 打开 experiments.json，只修改 day16.max_attempts 为 2，保持其他键不变并保存。

- 执行 python -m json.tool experiments.json 检查语法，再执行 python run.py day 16。

- 对比两份结果，重点观察retry.attempts、retry.status 和 deadline.status；解释改变原因后恢复原设置或明确记录保留值。

观察什么：默认成功前失败2次；将总尝试上限降为2后，应停止并给出耗尽或失败状态，而不是无限等待。

机制解释：重试上限是总尝试次数，第一次也计入。明确计数规则能避免越界与错误的可用性预期。

常见问题

修改配置后结果没有变化

原因：改了其他天的字段，或编辑后尚未保存。

处理：核对 experiments.json 中的 day16，保存后重新运行；检查实际执行目录。

把受控失败误当成程序坏了

原因：本课实验主动制造了边界条件。

处理：先对照 expected 和状态字段，区分设计内拒绝与未处理异常；保留错误信息再修复。

03 · 验收

区分暂时故障、参数错误与时间预算，避免无限重试。 保留基线、修改结果与自己的解释。

成果文件：outputs/day16.json

☐ 能用参数预测成功或耗尽。

☐ 保留成功与失败两份输出，并指出停止原因。

☐ 写明一个不能盲目重试的有副作用动作。

自测与解析

1. 成功前失败2次，总尝试上限2次，应该怎样结束？

- A. 第三次再试一次

- B. 报告失败并停止

- C. 把错误忽略后报告成功

- D. 无限重试直到成功

答案：B。 第三次超出了明确预算。受控失败是正确行为，不能为了成功率悄悄增加次数。

2. 等待超时是否必然意味着底层工作已停止？

- A. 必然停止

- B. 一定回滚所有副作用

- C. 取决于执行与取消机制

- D. 意味着输入必定错误

答案：C。 超时、取消与回滚是不同机制，需要分别设计和验证。

小词典

- Retry：在明确条件与次数限制内重新尝试操作。

- Deadline：某次工作可消耗的时间边界。

- Side effect：写文件、发送请求等会改变外部状态的效果。

04 · 我的记录

- 我的任务遇到哪类错误应该立即停止?

- 如果没有收到响应，我怎样判断写入是否已经发生?

忙碌日：20分钟

- 用5分钟阅读第一个概念，写出本课要解决的问题。

- 用10分钟运行 python run.py day 16，打开 outputs/day16.json，检查一个关键字段。

- 用5分钟完成第一道自测并留一句解释，完整实验与验收留到下次。

保存一份运行结果和自己的解释；精简练习单独记录，不等于整课验收全部完成。

有余力再做：20分钟选学 · 把本课机制迁移到一个自己的任务

- 从自己的文献整理或数据分析流程中选一个相似环节。

- 写清输入、允许动作、失败出口与验收方法。

- 提出一个会让当前实现失败的反例，并标明它是待实现扩展。

参考资料与阅读范围

Python：concurrent.futures：阅读 Future.result(timeout) 与 Executor 的关闭行为。

https://docs.python.org/3/library/concurrent.futures.html

BioAgentLab：科研 Agent 教材：查阅运行失败、恢复与可靠性相关章节，按本课失败类型对照阅读。

https://github.com/zbyloveyj/BioAgentLab

下一步

下一课把写入前审阅与幂等执行落实到一个可检查的文件。